Trường Đại học Nguyễn Tất Thành
Khoa Công Nghệ Thông Tin

KỸ THUẬT LẬP TRÌNH

TS. Võ Thị Hồng Thắm

CHƯƠNG III.
CLASS & OBJECT

NỘI DUNG

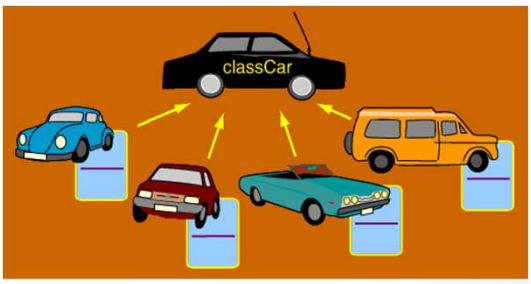
1. Khái niệm class & object
2. Ưu điểm của OOP
3. Khai báo class
4. Phạm vi truy xuất
5. Hàm tạo, get, set
6. Từ khóa this, kỹ thuật overloading

HĐ: Trả lời các câu hỏi sau vào tập (10 phút), thảo luận với bạn bên cạnh (10 phút) và đưa ra câu trả lời trước lớp

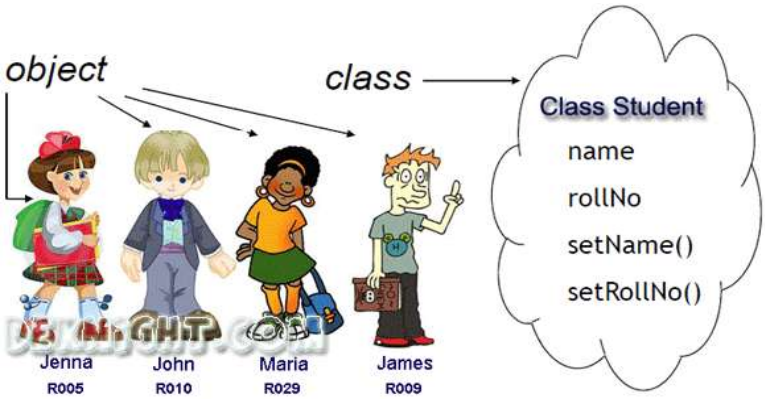
- Class là gì?
- Object là gì?
- Cho ví dụ minh họa

Khái niệm class và object

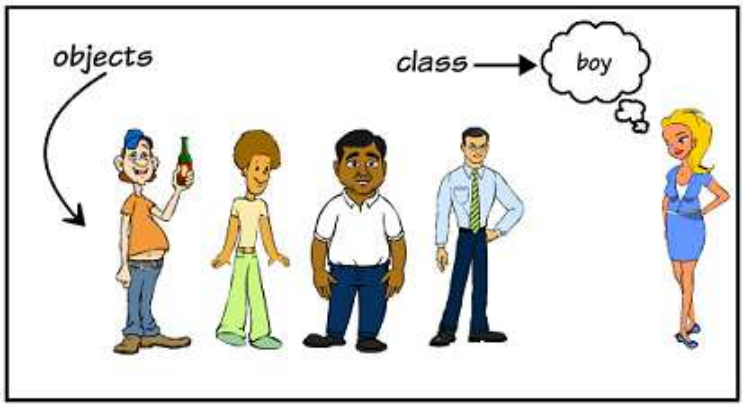
- Class là tập hợp các thuộc tính (*properties*) cùng với những phương thức (*methods*) đặc trưng cho một loại đối tượng nào đó cần dùng đến trong chương trình
- Trong lập trình hướng đối tượng, người ta ví class như là những "khuôn mẫu" dùng để "đúc" ra những đối tượng (*objects*) cho chương trình



Minh hoạ cho Class - Objects



Minh hoạ cho Class - Objects



HĐ: Anh/chị trả lời câu hỏi sau

- Ưu điểm của lập trình theo hướng đối tượng là gì?

Một số ưu điểm của OOP

- Thiết kế chương trình theo hướng trừu tượng – **Abstraction**
- Đóng gói và che giấu thông tin – **Encapsulation**
- Có thể thừa kế (*Thừa hưởng, di truyền*) từ một lớp có sẵn - **Inheritance**
- Năng cao khả năng tái sử dụng mã nguồn – **Reusable**

Cú pháp khai báo class

```
class <<ClassName>>
{
    <<type>> <<field1>>;
    ...
    <<type>> <<fieldN>>;

    <<type>> <<method1>>([parameters]) {
        // body of method
    }
    ...
    <<type>> <<methodN>>([parameters]) {
        // body of method
    }
}
```

Khai báo các trường

Khai báo các phương thức

Khai báo Class Employee

```
public class Employee{
    public String fullname;
    public double salary;

    public void input(){
        Scanner scanner = new Scanner(System.in);
        System.out.print(" >> Full Name: ");
        this.fullname = scanner.nextLine();

        System.out.print(" >> Salary: ");
        this.salary = scanner.nextDouble();
    }

    public void output(){
        System.out.println(this.fullname);
        System.out.println(this.salary);
    }
}
```

Trường

Phương thức

Lớp Employee có 2 thuộc tính là fullname và salary và 2 phương thức là input() và output()

Tạo & sử dụng đối tượng

- Tạo đối tượng của 1 class đã định nghĩa
- Cú pháp
  <className> <object> = **new** <className>();

- Truy xuất đến thành viên thuộc class
- Cú pháp
  <object>.<thuocTinh>;
  <object>.<phuongThuc>();

Tạo vào sử dụng object

```
public static void main(String[] args) {

    Employee emp = new Employee();
    emp.input();
    emp.output();
}
```

- Chú ý:
  - Toán tử **new** được sử dụng để tạo đối tượng
  - Biến emp chứa tham chiếu tới đối tượng
  - Sử dụng dấu chấm (.) để truy xuất các thành viên của lớp (trường và phương thức).

Bài tập tạo class

- Khai báo class **sinhVien (MSSV,HoTen,GioiTinh,DienThoai,DiemTK)**
- Khai báo class **hangHoa (MaHang, TenHang, DonViTinh,GiaBan,SoLuong)**
- Khai báo class **phuongTienGiaoThong**
- Khai báo class **nhanVien (MSNV,HoTen,Luong)**

Hàm khởi tạo - Constructor

- **Constructor** là gì ?
- Tại sao phải có **constructor**
- Khi nào nên sử dụng **constructor**
- Có bao nhiêu loại **constructor**
- **Constructor** được gọi khi nào ?

Một số lưu ý khi xây dựng constructor

- Tên của **Constructors** phải trùng với tên của lớp mà nó trực thuộc
- **Constructors** phải được khai báo **public** và không có kiểu trả về, đồng thời cũng không được sử dụng khai báo **void** ở phía trước tên hàm
- Có thể định nghĩa nhiều **Constructors** khác nhau theo nguyên tắc về hàm nạp chồng (Function Overloading)
- Trong trường hợp có nhiều **Constructors** được định nghĩa cho lớp thì khi khai báo các đối tượng của lớp đó, ta phải chỉ ra các tham số sao cho phù hợp với từng **Constructors** muốn sử dụng để trình biên dịch có thể nhận biết và phân biệt giúp cho việc gọi chính xác **Constructors** theo yêu cầu của người lập trình.

Overloading method

- Trong một lớp, ta có thể định nghĩa nhiều phương thức có cùng tên với nhau, nhưng các tham số của các phương thức này phải khác nhau (*Khác kiểu, khác số lượng, thứ tự tham số*)
- VD:

```
public class MyClass{
    void method(){...}
    void method(int x){...}
    void method(float x){...}
    void method(int x, double y){...}
}
```

Ví dụ :: Overloading method

```
class MayTinh{
    int tong(int a, int b){return a + b;}
    int tong(int a, int b, int c){return a + b + c;}
}
```

```
MayTinh mt = new MayTinh();
int t1 = mt.tong(5, 7);
int t2 = mt.tong(5, 7, 9);
```

Từ khóa this

- Đại diện cho đối tượng hiện tại của class
- Sử dụng để tham chiếu đến các thành viên của class một cách tường minh, tránh bị nhầm lẫn với các biến khác. VD tránh bị trùng với tham số của hàm

```
public class MyClass{
    int field;
    void method(int field){
        this.field = field;
    }
}
```

Trường

Tham số

Định nghĩa class sinhVien

SinhVien
+ hoTen: String + diemTB: double
+ xepLoai(): String + xuat(): void + nhap(): void
+ SinhVien() + SinhVien(hoTen, diemTB)

Modifier :: Mô tả phạm vi truy xuất

- **private**: chỉ được phép sử dụng nội bộ trong class
- **public**: công khai hoàn toàn
- **{default}**:
  - Là public đối với các lớp truy xuất cùng gói
  - Là private với các lớp truy xuất khác gói.
- **protected**: tương tự {default} nhưng cho phép kế thừa dù lớp con và cha khác gói.

Encapsulation :: Tính đóng gói

- **Ecapsulation** là đặc tính cho phép che giấu thông tin trong hướng đối tượng. Việc che giấu giúp cho việc bảo vệ dữ liệu thành viên, tránh những can thiệp không mong muốn từ bên ngoài class, đồng thời tăng cường khả năng độc lập dữ liệu giữa đối tượng của lớp với bên ngoài môi trường.
- Nếu không che giấu thông tin thì sao ?!. Hãy xem xét 1 ví dụ sau

```
public class SinhVien{
    public String hoTen;
    public double diem;
}
```

```
public class MyClass{
    public static void main(String[] args){
        SinhVien sv = new SinhVien();
        sv.hoTen = "Nguyễn Văn Tèo";
        sv.diem = 20.5;
    }
}
```

Encapsulation :: Tính đóng gói

- Để che giấu thông tin, nên sử dụng từ khóa private cho các field là dữ liệu thành viên của lớp.
- VD:
  private String hoTen;
  private double diem;
- Đã che giấu rồi thì làm sao truy cập từ bên ngoài để đọc hoặc gán dữ liệu ?!. Nên bổ sung các phương thức getter, setter để cho phép truy xuất tới dữ liệu đã bị che giấu trong các trường hợp đọc, ghi dữ liệu cho đối tượng của lớp. VD đối với thuộc tính hoTen, ta xây dựng các getter, setter có dạng như sau:

```
public String getHoTen(){
    return  this.hoTen;
}
public void setHoTen(String s){
    this.hoTen = s;
}
```

Example:: Xây dựng class sinhVien

```
public class SinhVien{
    private String hoTen;
    private double diem;
    public void setHoTen(String hoTen){
        this.hoTen = hoTen;
    }
    public String getHoTen(){
        return this.hoTen;
    }
    public void setDiem(double diem){
        if(diem < 0 || > 10){
            System.out.println("Điểm không hợp lệ");
        }
        else{
            this.diem = diem;
        }
    }
    public String getDiem(){
        return this.diem;
    }
}
```

```
public class MyClass{
    public static void main(String[] args){
        SinhVien sv = new SinhVien();
        sv.setHoTen("Nguyễn Văn Tèo");
        sv.setDiem(20);
    }
}
```

Package

- Package được sử dụng để tổ chức mã nguồn theo nhóm có cùng ý nghĩa, cùng mục đích. Điều này giúp cho việc quản lý mã nguồn được thuận tiện & khoa học. Nhất là trong trường hợp mã nguồn có khối lượng rất lớn các class trong những project thực tế. Có thể thấy khái niệm package tương tự như khái niệm folder hay directory của hệ điều hành dùng cho việc tổ chức quản lý tập tin ở trên đĩa.
- Java tổ chức rất nhiều pakage khác nhau để chứa mã nguồn thực thi như:
  - ❖ java.util
  - ❖ java.io
  - ❖ java.lang
  - ❖ ...

Package

- Vậy trong project của mình, chúng ta có thể tạo package được hay không ?. Sau khi tạo pakage thì khai báo sử dụng như thế nào ?
- Bạn hoàn toàn
khóa pakage tr

helloWorld.java

```
package firstProgram;

public class helloWorld {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        System.out.println("This is my first java program");
        System.out.println("--- Hello world !!! ---------");
    }

}
```

Khai báo pakage cho class helloWorld

Package

- Để sử dụng các mã nguồn thuộc 1 pakage
nào
the
imp
- Tro
pac
của
Bu
phím

*inputStream.java

```
package firstProgram;

import java.io.*;

public class inputStream {

    public static void main(String[] args) {
        try {
            //--- Tạo đối tượng đại diện cho bàn phím
            InputStreamReader reader = new InputStreamReader(System.in);
            //--- Tạo bộ đệm để nhận dữ liệu bàn phím
            BufferedReader buff = new BufferedReader(reader);
            //--- Sử dụng phương thức readLine để đọc 1 dòng dữ liệu
            String s = buff.readLine();
            System.out.print("Xin chào \"" + s +"\", rất vui vì bạn đã đọc được dữ liệu !.");
        } catch (IOException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
    }

}
```

Khai báo gói java.io để sử dụng trong chương trình

Bạn đã hiểu?

- Class và Object trong Java là gì? Class trong Java là một khuôn mẫu (template/blueprint) để định nghĩa các đối tượng. Nó chứa các thuộc tính (attributes) và phương thức (methods) để định nghĩa hành vi của đối tượng. Object trong Java là một thực thể (instance) của một class, được tạo ra bằng cách sử dụng từ khóa "new" và thực hiện các phương thức của class đó.
- Sự khác nhau giữa Class và Object trong Java là gì? Class là một khuôn mẫu (template) để định nghĩa các đối tượng trong khi Object là một thực thể (instance) của class được tạo ra bằng cách sử dụng từ khóa "new".

Bạn đã hiểu?

- Cách khởi tạo một Object trong Java? Để khởi tạo một Object trong Java, ta sử dụng từ khóa "new" sau đó là tên của class và các tham số của constructor nếu có. Ví dụ:

```
TenLop tenBien = new TenLop();
```

Bạn đã hiểu?

- Constructor trong Java là gì? Constructor là một phương thức đặc biệt trong một class được sử dụng để khởi tạo Object của class đó. Constructor có cùng tên với class và không có kiểu trả về. Nó được gọi tự động khi Object được tạo.

Bạn đã hiểu?

- Sự khác nhau giữa Constructor và Method trong Java là gì? Constructor và Method đều là phương thức trong Java. Tuy nhiên, Constructor có cùng tên với class, không có kiểu trả về và được gọi tự động khi Object được tạo. Trong khi đó, Method không có tên giống với class, có kiểu trả về và được gọi bằng tên và các tham số của nó.

Bạn đã hiểu?

- Getter và Setter Method trong Java là gì? Getter và Setter Method là hai phương thức được sử dụng để truy cập và cập nhật giá trị của thuộc tính trong class. Getter Method được sử dụng để lấy giá trị của thuộc tính trong khi Setter Method được sử dụng để cập nhật giá trị của thuộc tính.

Bạn đã hiểu?

- Access Modifier trong Java là gì? Access Modifier trong Java là từ khóa được sử dụng để quản lý quyền truy cập đến các thành phần của một class. Có bốn loại Access Modifier trong Java:
  - Public cho phép truy cập từ mọi nơi,
  - Private chỉ cho phép truy cập từ bên trong class,
  - Protected cho phép truy cập từ bên trong class và các class con, và
  - Default cho phép truy cập từ bên trong package.

Bài tập

1. Tạo 1 project với 2 class theo yêu cầu sau đây:
   a. Xây dựng class **HinhTron** với các yêu cầu:
      i. Khai báo thuộc tính bán kính
      ii. Các phương thức phục vụ cho việc tính diện tích, chu vi và nhập dữ liệu cho hình tròn:
         - ✓ **dienTich()**
         - ✓ **chuVi()**
         - ✓ **nhapDl()**

   b. Tạo class **ThiHanh**, với **main** thực hiện tạo đối tượng của lớp **HInhTron** rồi tiến hành tính toán các thông số cần thiết cho hình tròn

**** Có 2 hàm constructor : HinhTron() và HinhTron (float banKinh)***
**** Có hàm getter và setter***
**** Có 2 hàm nhapDl (Một hàm có tham số, một hàm không tham số nhập từ bàn phím)***
**** Trong main tạo 2 object HT1 và HT2 bằng 2 hàm tạo (constructor) khác nhau đã khai báo***


```java
public class HinhTron {
    private double banKinh;
    // Phương thức nhập dữ liệu
    public void nhapDl() {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Nhập bán kính: ");
        this.banKinh = scanner.nextDouble();
    }
    // Phương thức tính diện tích
    public double dienTich() {
        return Math.PI * banKinh * banKinh;
    }
    // Phương thức tính chu vi
    public double chuVi() {
        return 2 * Math.PI * banKinh;
    }
    // Phương thức hiển thị thông tin hình tròn
    public void hienThi() {
        System.out.println("Bán kính: " + banKinh);
        System.out.println("Chu vi: " + chuVi());
        System.out.println("Diện tích: " + dienTich());
    }
    // Hàm main để chạy thử lớp HinhTron
    public static void main(String[] args) {
        HinhTron hinhTron = new HinhTron();
        hinhTron.nhapDl();
        hinhTron.hienThi();
    }
}
```

```java
import java.util.Scanner;

public class HinhTron {
    private double banKinh;

    // Constructor mặc định
    public HinhTron() {
        this.banKinh = 0;
    }

    // Constructor có tham số
    public HinhTron(double banKinh) {
        this.banKinh = banKinh;
    }
```



```java
    // Hàm main để chạy thử lớp HinhTron
    public static void main(String[] args) {
        HinhTron hinhTron = new HinhTron(3.5);
        hinhTron.hienThi();
    }
}
```

Bài tập

2. Tạo 1 project với 2 class theo yêu cầu sau đây:
   a. Xây dựng class **HinhChuNhat** với các yêu cầu:
      i. Khai báo các thuộc tính chieuDai, chieuRong
      ii. Các phương thức phục vụ cho việc tính diện tích, chu vi và nhập dữ liệu cho hình chữ nhật:
         - ✓ **dienTich()**
         - ✓ **chuVi()**
         - ✓ **nhapDl()**

**** Thêm 2 hàm constructor : HinhChuNhat() và HinhChuNhat (float cD, float cR). Hàm không tham số gọi thự thi hàm nhapDl***

- ***Thêm hàm getter và setter cho chieuDai và chieuRong***
- ***Thêm hàm inKetQua: in chu vi và diện tích ra màn hình***
- ***Hàm main (ở class ThiHanh) tạo 3 object HCN1, HCN2 và HCN3:***
  - ***Bằng hàm constructor không tham số***
  - ***Bằng hàm constructor có tham số***
  - ***In chuVi, dienTich ra màn hình bằng cách gọi hàm inKetQua***


```java
import java.util.Scanner;
public class HinhChuNhat {
    private double chieuDai;
    private double chieuRong;
    // Phương thức nhập dữ liệu từ bàn phím
    public void nhapDl() {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Nhập chiều dài: ");
        this.chieuDai = scanner.nextDouble();
        System.out.print("Nhập chiều rộng: ");
        this.chieuRong = scanner.nextDouble();
    }
    // Phương thức tính diện tích
    public double dienTich() {
        return chieuDai * chieuRong;
    }
    // Phương thức tính chu vi
    public double chuVi() {
        return 2 * (chieuDai + chieuRong);
    }
    // Phương thức hiển thị thông tin hình chữ nhật
    public void hienThi() {
        System.out.println("Chiều dài: " + chieuDai);
        System.out.println("Chiều rộng: " + chieuRong);
        System.out.println("Chu vi: " + chuVi());
        System.out.println("Diện tích: " + dienTich());
    }
    // Hàm main để chạy thử lớp HinhChuNhat
    public static void main(String[] args) {
        HinhChuNhat hcn = new HinhChuNhat();
        hcn.nhapDl();
        hcn.hienThi();
    }
}
```

Bài tập

3. Tạo 1 project với 2 class theo yêu cầu sau đây:
   a. Xây dựng class **TamGiac** với các yêu cầu:
      i. Khai báo các thuộc tính canhA, canhB, canhC
      ii. Các phương thức phục vụ cho việc tính diện tích, chu vi và nhập dữ liệu cho hình tam giác:
         - ✓ **dienTich**()
         - ✓ **chuVi**()
         - ✓ **nhapDl**()

   b. Tạo class **ThiHanh**, với **main** thực hiện tạo đối tượng của lớp **TamGiac** và tiến hành tính toán các thông số cần thiết cho hình tam giác

Bài tập

5. Định nghĩa class **SinhVien** với các yêu cầu sau:
   a. Các thuộc tính dùng cho lưu trữ thông tin:
      i. Mã số sinh viên
      ii. Họ tên sinh viên
      iii. Giới tính
      iv. Số điện thoại
      v. Điểm tổng kết

   b. Xây dựng các phương thức
      i. **nhapDl**(): cho phép nhập thông tin của sinh viên
      ii. **xepLoai**(): xếp loại học sinh dựa trên điểm trung bình
      iii. **thongTin**(): Hiển thị thông tin của sinh viên
   c. Xây dựng class **ThiHanh** rồi tạo đối tượng của lớp **SinhVien** & thi hành các phương thức của lớp đã tạo

Bài tập

6. Định nghĩa class **SanPham** với các yêu cầu sau:
   a. Các thuộc tính dùng cho lưu trữ thông tin:
      i. Mã sản phẩm
      ii. Tên sản phẩm
      iii. Đơn vị tính
      iv. Giá bán
      v. Số lượng
   b. Xây dựng các phương thức
      i. **nhapTt**(): cho phép nhập thông tin của cản phẩm
      ii. **triGia**(): Tính trị giá của sản phẩm dựa theo số lượng và giá bán
      iii. **vat**(): Tính thuế giá trị gia tăng là 10% trị giá của sản phẩm tương ứng
      iv. **thanhTien**(): tính thành tiền dựa trên trị giá và thuế VAT
      v. **thongTin**(): Hiển thị thông tin của sản phẩm
   c. Xây dựng class **ThiHanh** rồi tạo đối tượng của lớp **SanPham** & thi hành các phương thức của lớp đã tạo

```java
import java.util.Scanner;

public class SanPham {
    private String maSanPham;
    private String tenSanPham;
    private String donViTinh;
    private double giaBan;
    private int soLuong;

    // Constructor mặc định
    public SanPham() {
        this.maSanPham = "";
        this.tenSanPham = "";
        this.donViTinh = "";
        this.giaBan = 0;
        this.soLuong = 0;
    }

    // Constructor có tham số
    public SanPham(String maSanPham, String tenSanPham, String donViTinh, double giaBan, int soLuong) {
        this.maSanPham = maSanPham;
        this.tenSanPham = tenSanPham;
        this.donViTinh = donViTinh;
        this.giaBan = giaBan;
        this.soLuong = soLuong;
    }
```

```java
// Phương thức nhập thông tin sản phẩm
    public void nhapTt() {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Nhập mã sản phẩm: ");
        this.maSanPham = scanner.nextLine();
        System.out.print("Nhập tên sản phẩm: ");
        this.tenSanPham = scanner.nextLine();
        System.out.print("Nhập đơn vị tính: ");
        this.donViTinh = scanner.nextLine();
        System.out.print("Nhập giá bán: ");
        this.giaBan = scanner.nextDouble();
        System.out.print("Nhập số lượng: ");
        this.soLuong = scanner.nextInt();
    }

    // Phương thức tính trị giá
    public double triGia() {
        return giaBan * soLuong;
    }

    // Phương thức tính VAT (10% trị giá)
    public double vat() {
        return triGia() * 0.1;
    }
```



```java
    // Phương thức tính thành tiền (trị giá + VAT)
    public double thanhTien() {
        return triGia() + vat();
    }

    // Phương thức hiển thị thông tin sản phẩm
    public void thongTin() {
        System.out.println("Mã sản phẩm: " + maSanPham);
        System.out.println("Tên sản phẩm: " + tenSanPham);
        System.out.println("Đơn vị tính: " + donViTinh);
        System.out.println("Giá bán: " + giaBan);
        System.out.println("Số lượng: " + soLuong);
        System.out.println("Trị giá: " + triGia());
        System.out.println("VAT (10%): " + vat());
        System.out.println("Thành tiền: " + thanhTien());
    }

    // Hàm main để chạy thử lớp SanPham
    public static void main(String[] args) {
        SanPham sp = new SanPham();
        sp.nhapTt();
        sp.thongTin();
    }
}
```

Câu hỏi

- Đoạn code sau in ra gì?

```
public class Student {
    private String name;

    public String getName() {
        return name;
    }

    public void setName(String newName) {
        name = newName;
    }
}
```

```
Student s = new Student();
s.setName("Alice");
System.out.println(s.getName());
```

45

Chương trình tránh điều gì?

```
public class BankAccount {
    private double balance;

    public double getBalance() {
        return balance;
    }

    public void setBalance(double balance) {
        this.balance = balance;
    }
}
```

```
public void setBalance(double balance) {
    if (balance >= 0) {
        this.balance = balance;
    }
}
```

46

Kết quả?

```java
public class Person {
    private int age;

    public int getAge() {
        return age;
    }

    public void setAge(int newAge) {
        if (newAge > 0) {
            age = newAge;
        }
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        Person p = new Person();
        p.setAge(-5);
        System.out.println(p.getAge());
    }
}
```


setter & getter

- Giúp đóng gói (**encapsulation**) và kiểm soát truy cập dữ liệu trong lớp

Trường Đại học Nguyễn Tất Thành
Khoa Công Nghệ Thông Tin

-Hết chương 3-